UNIVERSITÉ PROTESTANTE AU CONGO
FACULTÉ DES SCIENCES INFORMATIQUES
KINSHASA/LINGWALA

Travail Pratique

SYSTÈME DE GESTION DE FACTURATION EN PHP

Matière : PROGRAMMATION WEB PHP

Conçu par les étudiants :
1. KAGHOMA KITSIPA John
2. KIPAKA BAKARI Michel
3. MATHE MWERO Ben

Promotion : L2 LMD

Année Académique : 2025-2026

Table des matières

1 Introduction Générale **3**

1.1 Contexte et Justification du Projet . . . 3

1.2 Problématique . . . 3

1.3 Objectifs du Travail . . . 4

1.4 Méthodologie de Réalisation . . . 4

1.5 Analyse de l'Architecture Logicielle . . . 4

1.5.1 Choix des Technologies . . . 4

1.5.2 PHP (Hypertext Preprocessor) . . . 4

1.5.3 JSON (JavaScript Object Notation) . . . 5

1.5.4 Vanilla CSS et JavaScript . . . 5

1.5.5 Organisation des Fichiers . . . 5

1.5.6 Le dossier /config . . . 5

1.5.7 Le dossier /data . . . 5

1.5.8 Le dossier /includes . . . 5

1.5.9 Le dossier /modules . . . 6

2 Gestion et Structures de Données **7**

2.1 Philosophie du Stockage Flat-File . . . 7

2.2 Modélisation des Entités . . . 7

2.2.1 Entité Utilisateur . . . 7

2.2.2 Entité Produit . . . 7

2.2.3 Entité Facture . . . 7

2.3 Justification de la Sérialisation JSON . . . 8

3 Description et Logique des Modules **9**

3.1 Module d'Authentification . . . 9

3.1.1 Gestion des Sessions . . . 9

3.1.2 Processus de Login . . . 9

3.2 Module de Facturation et Panier . . . 9

3.2.1 Le Panier Virtuel . . . 9

3.2.2 Algorithme de calcul . . . 9

3.3 Module de Gestion des Produits . . . 10

4 Interface Utilisateur et Ergonomie **11**
4.1 Principes de Design UI 11
4.2 Intégration du Scanner de Codes-Barres 11
4.3 Diagramme de Flux 11

5 Manuel d'Utilisation du Système **13**
5.1 Démarrage et Connexion 13
5.2 Opération de Vente (Profil Caissier) 13
5.3 Administration du Stock (Profil Admin) 13
5.4 Difficultés Rencontrées et Solutions 14
5.4.1 Gestion des conflits d'écriture 14
5.4.2 Compatibilité du scanner QuaggaJS 14
5.4.3 Précision des calculs flottants 14

6 Tests, Validation et Sécurité **15**
6.1 Protocole de Test 15
6.2 Sécurité du Système 15

7 Conclusion et Perspectives **16**
7.1 Bilan du Travail 16
7.2 Pistes d'Amélioration 16
7.3 Glossaire Technique 16

8 Annexes : Extraits de Code Source **18**
8.1 Logique de Configuration (`config.php`) 18
8.2 Gestion des Produits (`fonctions-produits.php`) 18

Chapitre 1

Introduction Générale

1.1 Contexte et Justification du Projet

L'ère numérique actuelle impose aux entreprises, quelle que soit leur taille, une mutation profonde de leurs processus opérationnels. Dans le contexte de la République Démocratique du Congo, et plus particulièrement à Kinshasa, la gestion des points de vente reste un défi majeur pour de nombreux entrepreneurs. La transition vers des outils numériques est souvent freinée par la complexité des solutions existantes ou par les coûts d'infrastructure.

Le présent projet, réalisé dans le cadre du cours de Programmation Web PHP, vise à concevoir une solution de facturation légère, robuste et accessible. Le choix s'est porté sur une architecture basée sur PHP pour la logique serveur et JSON pour le stockage des données. Cette approche permet de s'affranchir de la lourdeur d'un serveur de base de données relationnelle comme MySQL, tout en conservant une structure de données organisée et persistante.

1.2 Problématique

La problématique centrale de ce travail réside dans la fiabilisation de l'acte de vente en magasin. Les méthodes de facturation manuelles souffrent de plusieurs lacunes critiques :

- **Erreurs de calcul :** L'application manuelle des prix et des taxes (TVA) est source fréquente d'erreurs comptables.
- **Lenteur opérationnelle :** En période d'affluence, la rédaction manuelle de factures pénalise la fluidité du service client.
- **Manque de traçabilité :** Sans archive numérique, il est extrêmement complexe de réaliser des statistiques de vente ou de retrouver une facture ancienne.
- **Sécurité des données :** Les registres papier sont vulnérables aux dégradations physiques et à la perte d'informations.

1.3 Objectifs du Travail

L'objectif général est de mettre en place un "Système de Gestion de Facturation" (SGF) capable d'automatiser le processus de vente. De manière spécifique, le système doit :

1. Assurer une authentification sécurisée des utilisateurs selon leurs rôles.
2. Permettre une gestion dynamique du catalogue de produits.
3. Intégrer un module de lecture de codes-barres pour accélérer la saisie.
4. Calculer automatiquement les montants HT, les taxes et le TTC.
5. Sauvegarder chaque transaction de manière persistante dans des fichiers JSON.
6. Offrir une interface utilisateur ergonomique et responsive.

1.4 Méthodologie de Réalisation

Pour la réalisation de ce projet, nous avons adopté une méthodologie inspirée du cycle en V, adaptée aux contraintes d'un projet étudiant :

— **Analyse des besoins :** Identification des acteurs (Caissier, Admin) et de leurs besoins spécifiques.
— **Modélisation des données :** Définition des structures JSON pour représenter les objets métiers (Utilisateur, Produit, Facture).
— **Développement modulaire :** Séparation des fichiers selon leurs responsabilités (Logique, Interface, Données).
— **Tests et Intégration :** Vérification de chaque fonctionnalité en environnement local (WAMP).

1.5 Analyse de l'Architecture Logicielle

1.5.1 Choix des Technologies

Le choix des technologies est un pilier fondamental de tout projet informatique. Pour ce système de facturation, nous avons opté pour un trio classique mais puissant : PHP, JSON et Vanilla JavaScript.

1.5.2 PHP (Hypertext Preprocessor)

PHP a été choisi pour sa maturité et sa simplicité de déploiement. En tant que langage côté serveur, il permet de gérer efficacement les sessions utilisateurs, de manipuler les fichiers sur le disque et d'appliquer la logique métier nécessaire aux calculs financiers. Son typage dynamique et sa large bibliothèque de fonctions natives (notamment pour le JSON) en ont fait l'outil idéal.

1.5.3 JSON (JavaScript Object Notation)

Le choix du format JSON comme "base de données" est une décision stratégique pour ce TP. Contrairement à XML, le JSON est plus léger et s'intègre nativement avec PHP via les fonctions `json_encode` et `json_decode`. Il permet de stocker des objets complexes sous forme de chaînes de caractères, ce qui facilite grandement la portabilité du projet. Aucun serveur SQL n'est requis, ce qui rend l'application "Plug and Play".

1.5.4 Vanilla CSS et JavaScript

Pour l'interface, nous avons évité l'utilisation de frameworks lourds. Le CSS pur (Vanilla CSS) a été utilisé pour créer une identité visuelle unique et légère. JavaScript a été sollicité pour la gestion dynamique du scanner de codes-barres, permettant une interaction en temps réel entre la caméra de l'utilisateur et le serveur PHP.

1.5.5 Organisation des Fichiers

L'organisation du code source a été pensée pour faciliter la maintenance et l'évolution future du système. Nous avons adopté une structure modulaire où chaque dossier a un rôle précis.

1.5.6 Le dossier /config

Ce dossier contient le fichier `config.php`. C'est le "cœur" de la configuration de l'application. Il définit les constantes globales telles que le taux de TVA, le chemin vers les fichiers de données et l'URL de base du projet. Centraliser ces informations permet de modifier le comportement de toute l'application en éditant un seul fichier.

1.5.7 Le dossier /data

Il contient la persistance du système. Trois fichiers principaux y sont stockés : *utilisateurs.json*, *produits.json* et *factures.json*. Ces fichiers agissent comme des tables de base de données. Il est crucial que ce dossier possède les droits d'écriture pour le serveur PHP.

1.5.8 Le dossier /includes

Ce dossier regroupe les fonctions métier réutilisables. Au lieu de réécrire la logique de lecture des fichiers JSON sur chaque page, nous avons créé des fonctions telles que `lireProduits()` ou `sauvegarderFacture()` qui sont incluses via `require_once`. Cela respecte le principe DRY (Don't Repeat Yourself).

1.5.9 Le dossier /modules

C'est ici que se trouve la logique spécifique à chaque fonctionnalité. Le module de facturation gère la création de factures, tandis que le module produits gère le catalogue. Cette séparation permet à plusieurs développeurs de travailler simultanément sur différentes parties du projet sans conflits majeurs.

Chapitre 2

Gestion et Structures de Données

2.1 Philosophie du Stockage Flat-File

Le stockage dans des fichiers plats (flat-file) est une méthode alternative aux bases de données relationnelles. Dans notre système, chaque fichier JSON représente une entité. Cette approche simplifie la sauvegarde (un simple copier-coller du dossier *data*) mais impose une rigueur particulière dans la gestion des accès concurrents.

2.2 Modélisation des Entités

2.2.1 Entité Utilisateur

Un utilisateur est défini par ses identifiants et ses droits d'accès. La sécurité est assurée par le hachage des mots de passe.

— **identifiant** (String) : Identifiant unique de connexion.
— **mot_de_passe** (String) : Empreinte hachée via BCRYPT.
— **role** (String) : Définit si l'utilisateur est un Administrateur ou un Caissier.
— **actif** (Boolean) : Permet de désactiver un compte sans le supprimer.

2.2.2 Entité Produit

Le catalogue est le pivot du système. Chaque produit doit être identifiable de manière unique.

— **code_barre** (String) : Identifiant scannable unique.
— **designation** (String) : Nom commercial du produit.
— **prix_unitaire** (Float) : Prix de vente hors taxe.
— **stock** (Integer) : Quantité disponible en magasin.

2.2.3 Entité Facture

Une facture est un document complexe regroupant des données d'en-tête et une liste d'articles.

— **id_facture** (String) : Numéro unique généré selon le format FAC-AAAA-NNN.
— **date** (DateTime) : Horodatage de la transaction.
— **caissier** (String) : Nom de la personne ayant réalisé la vente.
— **articles** (Array) : Liste d'objets contenant la désignation, la quantité et le prix au moment de la vente.
— **totaux** (Object) : Calculs finaux (HT, TVA, TTC).

2.3 Justification de la Sérialisation JSON

L'utilisation de la fonction `json_encode($data, JSON_PRETTY_PRINT)` est essentielle. Le flag `PRETTY_PRINT` assure que le fichier reste lisible par un humain, ce qui est crucial lors des phases de débogage. Lors de la lecture, `json_decode($data, true)` transforme la chaîne en un tableau associatif PHP, permettant d'accéder aux données via des clés simples (ex : `$produit['prix_unitaire']`).

Chapitre 3

Description et Logique des Modules

3.1 Module d'Authentification

3.1.1 Gestion des Sessions

Le système utilise les sessions natives de PHP (`session_start()`). Une session permet de maintenir l'état de l'utilisateur entre les différentes requêtes HTTP. Lors d'une connexion réussie, nous stockons l'ID de l'utilisateur et son rôle.

3.1.2 Processus de Login

Le script de connexion suit les étapes suivantes :

1. Récupération des données POST du formulaire.
2. Chargement du fichier *utilisateurs.json*.
3. Recherche de l'utilisateur par son identifiant.
4. Vérification du mot de passe via `password_verify()`.
5. Si correct, initialisation des variables de session et redirection vers le dashboard.

3.2 Module de Facturation et Panier

3.2.1 Le Panier Virtuel

Le panier n'est pas stocké dans un fichier tant que la facture n'est pas validée. Il réside dans la session (`$_SESSION['panier']`). Cela permet d'ajouter ou de retirer des articles sans impacter les données permanentes, offrant ainsi une grande fluidité à l'utilisateur.

3.2.2 Algorithme de calcul

Le calcul de la facture suit une logique rigoureuse :

— Pour chaque article dans le panier : `Sous-total = Prix Unitaire * Quantité`.

— `Total HT = Somme de tous les sous-totaux`.

— `TVA = Total HT * (Taux TVA / 100)`.
— `Total TTC = Total HT + TVA`.

Ces calculs sont effectués à chaque ajout d'article pour maintenir un affichage en temps réel du montant total.

3.3 Module de Gestion des Produits

Ce module permet aux administrateurs de maintenir à jour le catalogue. Il utilise des fonctions CRUD simples. Une attention particulière est portée à l'unicité du code-barre : avant chaque ajout, le système scanne le fichier JSON pour vérifier qu'aucun autre produit ne possède le même code.

Chapitre 4

Interface Utilisateur et Ergonomie

4.1 Principes de Design UI

L'interface utilisateur (UI) a été conçue pour être "minimaliste". Dans un environnement de vente, la surcharge d'informations est une source d'erreurs. Nous avons privilégié :

— **Contrastes élevés :** Texte sombre sur fond clair pour une lisibilité maximale sous un éclairage de magasin.

— **Zones d'action larges :** Boutons de grande taille pour faciliter l'utilisation sur les écrans tactiles (tablettes).

— **Retours visuels :** Utilisation de couleurs (vert pour succès, rouge pour erreur) pour confirmer les actions de l'utilisateur.

4.2 Intégration du Scanner de Codes-Barres

L'intégration du scanner via la bibliothèque QuaggaJS est une valeur ajoutée majeure. Elle utilise l'API `getUserMedia` du navigateur pour accéder à la caméra.

1. **Initialisation :** Le script demande l'accès à la caméra.
2. **Scan :** Le flux vidéo est analysé en temps réel pour détecter des motifs de codes-barres (EAN-13 par exemple).
3. **Décodage :** Une fois le code identifié, une requête AJAX (fetch) est envoyée au serveur PHP pour récupérer les informations du produit.
4. **Mise à jour :** Si le produit est trouvé, le panier est rafraîchi sans recharger toute la page.

4.3 Diagramme de Flux

Le diagramme suivant détaille le cycle de vie d'une transaction de vente, de la détection du produit à l'archivage de la facture finale.

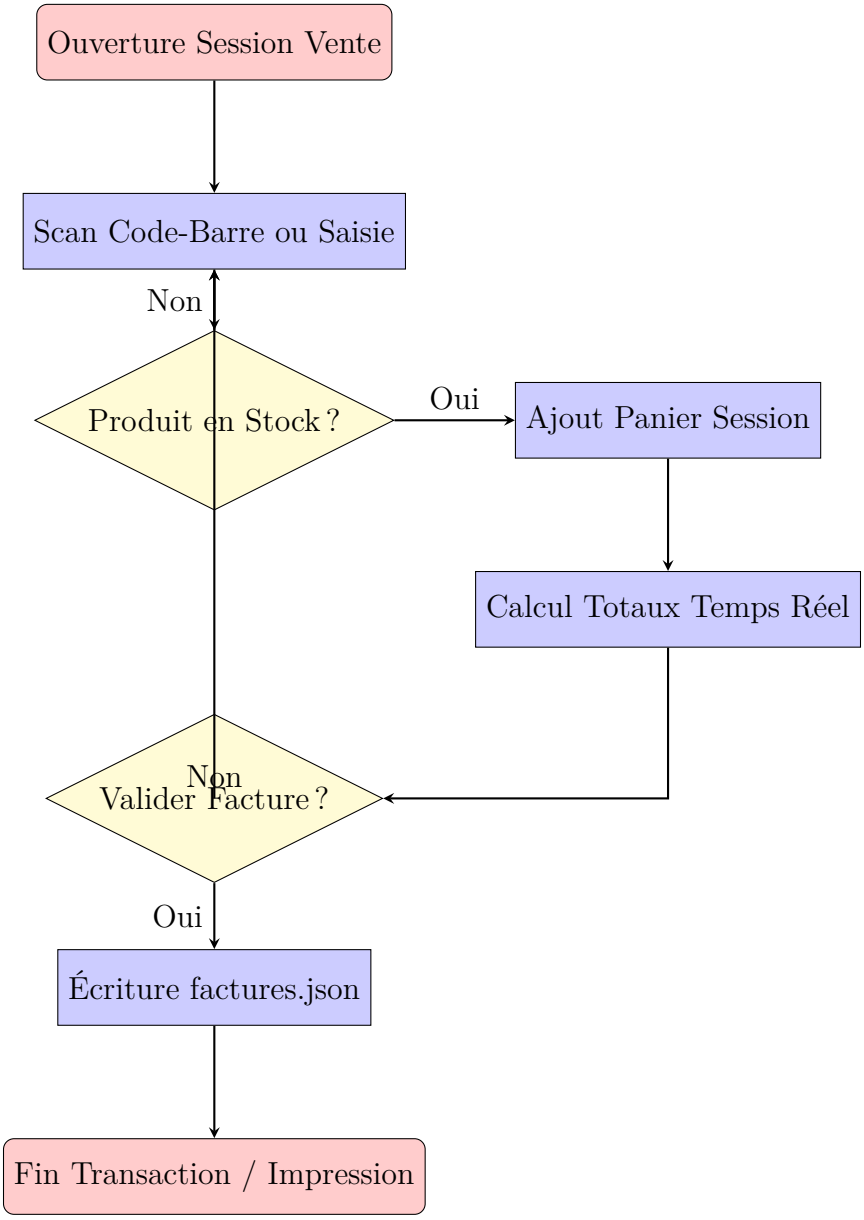

FIGURE 4.1 – Diagramme de flux d'une transaction de vente complète

Chapitre 5

Manuel d'Utilisation du Système

Ce chapitre détaille les procédures opérationnelles pour les différents profils d'utilisateurs.

5.1 Démarrage et Connexion

Pour lancer l'application, ouvrez votre navigateur et accédez à l'URL locale. Sur la page de garde :

1. Entrez votre nom d'utilisateur (ex : admin).
2. Saisissez votre mot de passe sécurisé.
3. Une fois connecté, vous arrivez sur le Tableau de Bord affichant les statistiques du jour (Chiffre d'affaires, nombre de factures).

5.2 Opération de Vente (Profil Caissier)

Le caissier passe la majeure partie de son temps sur le module "Nouvelle Facture".

1. Cliquez sur le bouton "Démarrer Scanner".
2. Présentez le code-barre du produit devant la caméra de l'ordinateur ou du smartphone.
3. Vérifiez que le produit s'affiche dans la liste en bas avec le prix correct.
4. Pour modifier une quantité, utilisez les boutons (+) ou (-) dans le tableau.
5. Une fois la liste terminée, cliquez sur "Valider la Facture". Un récapitulatif s'affiche pour confirmation.

5.3 Administration du Stock (Profil Admin)

Seuls les administrateurs ont accès à la gestion des produits.

— **Ajouter un produit :** Accédez à l'onglet "Produits", cliquez sur "Ajouter". Il est impératif de renseigner le code-barre exact figurant sur l'emballage.

— **Mise à jour des prix :** Vous pouvez modifier le prix d'un article existant. Notez que cette modification ne changera pas les factures déjà émises (les prix historiques sont conservés dans *factures.json*).

5.4 Difficultés Rencontrées et Solutions

5.4.1 Gestion des conflits d'écriture

Le principal défi d'un système basé sur des fichiers JSON est la concurrence. Si deux caissiers enregistrent une facture à la milliseconde près, l'un risque d'écraser les données de l'autre. **Solution :** Nous avons implémenté la fonction `flock()` de PHP qui permet de verrouiller un fichier pendant l'opération d'écriture, forçant les autres requêtes à attendre la fin du verrou.

5.4.2 Compatibilité du scanner QuaggaJS

Certains navigateurs restreignent l'accès à la caméra pour des raisons de sécurité (HTTPS requis). **Solution :** Pour les tests en local (HTTP), nous avons dû configurer les navigateurs pour accepter les sources de confiance locales et ajouter un mode de saisie manuelle au clavier en cas de défaillance matérielle de la caméra.

5.4.3 Précision des calculs flottants

PHP peut parfois arrondir les calculs de manière imprécise avec les nombres à virgule flottante. **Solution :** Nous utilisons systématiquement la fonction `number_format()` lors de l'affichage final pour garantir que les montants financiers respectent toujours deux décimales après la virgule.

Chapitre 6

Tests, Validation et Sécurité

6.1 Protocole de Test

Chaque fonctionnalité a été soumise à des tests unitaires et d'intégration.

— **Test de charge JSON :** Nous avons injecté 6 factures fictives pour vérifier que la lecture du fichier restait rapide sur un serveur standard.

— **Test d'intégrité :** Vérification qu'un produit dont le stock est à zéro ne peut pas être ajouté à une facture.

6.2 Sécurité du Système

— **Protection contre le vol de session :** L'ID de session est régénéré lors de chaque connexion.

— **Hachage BCRYPT :** Même en cas de vol du fichier *utilisateurs.json*, les mots de passe restent indéchiffrables grâce à la complexité de l'algorithme BCRYPT.

— **Validation des entrées :** Toutes les données provenant de formulaires sont nettoyées pour éviter les injections de scripts malveillants (XSS).

Chapitre 7

Conclusion et Perspectives

7.1 Bilan du Travail

Ce projet a été une opportunité exceptionnelle de mettre en pratique les notions théoriques abordées en cours de Programmation Web. Nous avons réussi à créer un système de facturation complet, léger et fonctionnel, capable de répondre aux besoins réels d'une petite entreprise. L'utilisation innovante du JSON nous a permis d'explorer les limites et les avantages des bases de données plates.

7.2 Pistes d'Amélioration

Le système actuel constitue une base solide mais pourrait évoluer vers :

— **Migration MySQL :** Pour supporter une volumétrie de données beaucoup plus importante.

— **Génération de PDF :** Création automatique de factures formatées pour l'impression via Dompdf.

— **Reporting Statistique :** Ajout de graphiques de vente mensuels.

— **Multi-sites :** Synchronisation des données entre plusieurs magasins via une API.

7.3 Glossaire Technique

AJAX : Asynchronous JavaScript and XML. Technique de développement web permettant de mettre à jour des parties d'une page web sans rechargement.

BCRYPT : Algorithme de hachage de mots de passe hautement sécurisé.

BCRYPT : Algorithme de hachage de mots de passe hautement sécurisé.

CSS : Cascading Style Sheets. Langage utilisé pour mettre en forme les pages web.

DRY : Don't Repeat Yourself. Principe de programmation visant à éviter la répétition de code.

HTTPS : HyperText Transfer Proto-

col Secure. Version sécurisée du protocole de transfert hypertexte.

JSON : JavaScript Object Notation. Format de données textuel léger et facile à échanger.

LMD : Licence, Master, Doctorat. Système d'organisation des études supérieures.

PHP : Hypertext Preprocessor. Langage de script côté serveur utilisé pour le développement web.

Responsive Design : Conception web permettant d'adapter l'affichage d'un site à la taille de l'écran.

TVA : Taxe sur la Valeur Ajoutée.

UPC : Université Protestante au Congo.

XSS : Cross-Site Scripting. Faille de sécurité permettant d'injecter des scripts malveillants dans une application.

Chapitre 8

Annexes : Extraits de Code Source

8.1 Logique de Configuration (`config.php`)

```php
<?php
define('TVA_RATE', 0.18);
define('DATA_PATH', __DIR__ . '/../data/');
define('PRODUITS_FILE', DATA_PATH . 'produits.json');
define('FACTURES_FILE', DATA_PATH . 'factures.json');
define('UTILISATEURS_FILE', DATA_PATH . 'utilisateurs.json');
```

8.2 Gestion des Produits (`fonctions-produits.php`)

```php
function chercherProduit($code_barre) {
    $produits = lireProduits();
    $code_barre = trim($code_barre);
    foreach ($produits as $produit) {
        if (trim($produit['code_barre']) === $code_barre) {
            return $produit;
        }
    }
    return null;
}
```